

第六单元：数组

1. 认识数组

2. 创建数组

2.1. 字面量

2.2. new Array

2.3. 数组空元素

3. 数组获取和设置

3.1. 数组的 length

3.2. 数组的索引，从 0 开始

4. 数组常用方法

4.1. push()

4.2. pop()

4.3. shift()

4.4. unshift()

4.5. splice()

4.6. slice()

4.7. concat()

4.8. indexOf()

4.9. lastIndexOf()

4.10. join()

4.11. reverse()

4.12. sort()

4.13. forEach()

4.14. map()

4.15. filter()

4.16. reduce() 归并

4.17. some()

4.18. every()

4.19. find()

4.20. `findIndex()`
4.21. `fill()`
4.22. `copyWithin()`
4.23. `at()`
4.24. `with()`
4.25. `flat()` 重构flat
4.26. `flatMap()`
4.27. `includes()`

5. 静态方法

5.1. `Array.isArray()`
5.2. `Array.from()`

6. 总结

1. 认识数组

- 数组是有序的数据集合。
- 数组中的值可以是任意类型。
- 每一个元素在数组中所处的位置，称之为索引。是数字来表示，从0开始。

2. 创建数组

2.1. 字面量

▼ 创建数组

JavaScript |

```
1 var arr1 = [] // 创建空数组
2 var arr2 = [1, 'b', true] // 按逗号分隔每一项
```

2.2. new Array

```
1 // 不传参数, 创建空数组
2 var arr1 = new Array() // []
3
4 // 传入一个数字, 创建一个指定长度的空数组
5 var arr2 = new Array(5) // 输出 length 为 5 的空数组
6
7 // 如果传递多个参数或一个非数字参数, 创建一个包含这些参数作为元素的新数组
8 var arr3 = new Array(1, 2, 3); // [1, 2, 3]
9 var arr4 = new Array('a'); // ['a']
10
```

2.3. 数组空元素

“空元素”通常指的是数组中存在的未定义（`undefined`）或者空字符串（`''`）元素

```
1 var arr=[,,3]
2 // for循环识别数组的空元素
3 for(var i=0;i<arr.length;i++){
4     console.log( i ) // 0 1 2 3
5     console.log( arr[i] ) // undefined undefined undefined 3
6     // 判断当前key【这里是下标】是否在对象arr中存在 可过滤空元素
7     if(i in arr){
8         console.log(arr[i]) //3
9     }
10 }
11 // forEach方法不会识别空元素
12 arr.forEach(item => console.log(item)) // 3
```

3. 数组获取和设置

3.1. 数组的 length

- `length` 返回数组的长度
- `length` 也可以设置

```
1 var arr = ['a', 'b', 'c', 'd']
2 console.log(arr.length) // 4
3 arr.length = 0 // 清空数组
4 console.log(arr) // []
```

3.2. 数组的索引，从 0 开始

```
1 var arr = ['a', 'b', 'c', 'd']
2 // 通过下标获取
3 console.log(arr[0]) // 获取第一项
4 console.log(arr[arr.length - 1]) // 获取最后一项
5
6 // 通过下标修改
7 arr[1] = 'BBB'
8 console.log(arr) // ['a', 'BBB', 'c', 'd']
```

4. 数组常用方法

4.1. push()

- 语法：array.push(element1, ..., elementN)
- 功能：在数组的末尾添加一个或多个元素。
- 返回值：新数组的长度。
- 是否改变原数组：是。
- 示例：

```
1 var arr = [1, 2, 3];
2 var result = arr.push(4); // arr变为[1, 2, 3, 4]
3 console.log(result); // 输出4
```

4.2. pop()

- 语法：array.pop()
- 功能：删除数组的最后一个元素。
- 返回值：被删除的元素。
- 是否改变原数组：是。
- 示例：

JavaScript |

```
1 var arr = [1, 2, 3, 4];
2 var lastElement = arr.pop(); // arr变为[1, 2, 3]
3 console.log(lastElement); // 输出4
```

4.3. shift()

- 语法：array.shift()
- 功能：删除数组的第一个元素。
- 返回值：被删除的元素。
- 是否改变原数组：是。
- 示例：

JavaScript |

```
1 var arr = [1, 2, 3];
2 var firstElement = arr.shift(); // arr变为[2, 3]
3 console.log(firstElement); // 输出1
```

4.4. unshift()

- 语法：array.unshift(element1, ..., elementN)
- 功能：在数组的开头添加一个或多个元素。
- 返回值：新数组的长度。
- 是否改变原数组：是。
- 示例：

```
1 var arr = [2, 3];
2 var newLength = arr.unshift(1); // arr变为[1, 2, 3]
3 console.log(newLength); // 输出3
```

4.5. splice()

- 语法：array.splice(start[, deleteCount[, item1[, item2[, ...]]]])
- 功能：在任意位置添加或删除元素，万能法。
- 返回值：包含被删除元素的数组。
- 是否改变原数组：是。
- 示例：

删除

```
1 var arr = [1, 2, 3, 4];
2 var removed = arr.splice(1, 2); // arr变为[1, 4]
3 console.log(removed); // 输出[2, 3]
```

添加

```
1 var arr = [1, 2, 4];
2 var removed = arr.splice(2, 0, 3); // arr变为[1, 2, 3, 4]
3 console.log(removed); // 输出[]
```

替换

```
1 var arr = [1, 2, 3, 4];
2 var removed = arr.splice(1, 2, 'a', 'b'); // arr变为[1, 'a', 'b', 4]
3 console.log(removed); // 输出[2, 3]
```

4.6. slice()

- 语法：array.slice([begin[, end]])
- 功能：返回数组的一部分，不修改原数组。
- 返回值：一个新数组，包含指定范围的元素。
- 是否改变原数组：否。

- 示例:

JavaScript |

```
1 var arr = [1, 2, 3, 4];
2 var newArr = arr.slice(1, 3); // newArr为[2, 3]
3 console.log(newArr); // 输出[2, 3]
```

4.7. concat()

- 语法: `array.concat(value1[, value2[, ...[, valueN]]])`
- 功能: 合并两个或多个数组。
- 返回值: 一个新数组, 包含原数组和其他数组/值的组合。
- 是否改变原数组: 否。
- 示例:

JavaScript |

```
1 var arr1 = [1, 2];
2 var arr2 = [3, 4];
3 var newArr = arr1.concat(arr2); // newArr为[1, 2, 3, 4]
4 console.log(newArr); // 输出[1, 2, 3, 4]
```

4.8. indexOf()

- 语法: `array.indexOf(searchElement[, fromIndex])`
- 功能: 返回数组中某元素的首个索引, 如果不存在则返回-1。
- 返回值: 索引 (整数) 或-1。
- 是否改变原数组: 否。
- 示例:

JavaScript |

```
1 var arr = [1, 2, 3, 4];
2 var index = arr.indexOf(3); // index为2
3 console.log(index); // 输出2
```

4.9. lastIndexOf()

- 语法: `array.lastIndexOf(searchElement[, fromIndex])`
- 功能: 返回数组中某元素的最后一个索引, 如果不存在则返回-1。
- 返回值: 索引 (整数) 或-1。
- 是否改变原数组: 否。
- 示例:

JavaScript

```
1 var arr = [1, 2, 3, 2, 1];
2 var index = arr.lastIndexOf(2); // index为3
3 console.log(index); // 输出3
```

4.10. join()

- 语法: `array.join([separator])`
- 功能: 将数组的所有元素连接成一个字符串。
- 返回值: 连接后的字符串。
- 是否改变原数组: 否。
- 示例:

JavaScript

```
1 var arr = ['Hello', 'World'];
2 var str = arr.join(' '); // str为'Hello World'
3 console.log(str); // 输出'Hello World'
```

4.11. reverse()

- 语法: `array.reverse()`
- 功能: 颠倒数组中元素的顺序。
- 返回值: 颠倒后的数组。
- 是否改变原数组: 是。
- 示例:


```

1  var arr = [1, 2, 3];
2  var reversed = arr.reverse(); // arr和reversed都变为[3, 2, 1]
3  console.log(reversed); // 输出[3, 2, 1]

```

4.12. sort()

- 语法：array.sort([compareFunction])
- 功能：对数组元素进行排序。
- 返回值：排序后的数组。
- 是否改变原数组：是。
- 示例：
 - 不传参数，将数组元素作为字符串进行排序，按照 ASCII 码值排序

不传参数

```

1  var arr = [3, 1, 4, 12, 5, 9];
2  var sorted = arr.sort(); // arr和sorted都变为 [1, 12, 3, 4, 5, 9]
3  console.log(sorted); // 输出[1, 12, 3, 4, 5, 9]

```

升序



```

1  var arr = [3, 1, 4, 12, 5, 9];
2  var sorted = arr.sort(function (a, b) {
3    return a - b
4  }); // arr和sorted都变为 [1, 3, 4, 5, 9, 12]
5  console.log(sorted); // 输出 [1, 3, 4, 5, 9, 12]

```

降序



```

1  var arr = [3, 1, 4, 12, 5, 9];
2  var sorted = arr.sort(function (a, b) {
3    return b - a
4  }); // arr和sorted都变为 [12, 9, 5, 4, 3, 1]
5  console.log(sorted); // 输出 [12, 9, 5, 4, 3, 1]

```

冒泡排序、选择排序、快排

说一下sort排序是如何利用冒泡排序规则，产生a,b

4.13. forEach()

- 语法：array.forEach(callback(currentValue, index, array))
- 功能：遍历数组的每个元素，执行回调函数。
- 返回值：无。
- 是否改变原数组：依回调函数而定。
- 示例：

```
JavaScript |  
1  var arr = [1, 2, 3];  
2  arr.forEach(item => console.log(item * 2)); // 输出2, 4, 6
```

4.14. map()

- 语法：array.map(callback(currentValue, index, array))
- 功能：创建一个新数组，其结果是该数组中的每个元素都调用一个提供的函数后的返回值。
- 返回值：新数组。
- 是否改变原数组：否。
- 示例：

```
JavaScript |  
1  var arr = [1, 2, 3];  
2  var doubled = arr.map(item => item * 2); // doubled为[2, 4, 6]  
3  console.log(doubled); // 输出[2, 4, 6]
```

4.15. filter()

- 语法：array.filter(callback(element, index, array))
- 功能：创建一个新数组，包含通过所提供函数实现的测试的所有元素。
- 返回值：新数组。
- 是否改变原数组：否。
- 示例：

```
1 var arr = [1, 2, 3, 4, 5];
2 var even = arr.filter(item => item % 2 === 0); // even为[2, 4]
3 console.log(even); // 输出[2, 4]
```

4.16. reduce() 归并

- 语法：array.reduce(callback(accumulator, currentValue, index, array), initialValue)
- 功能：对累加器和数组中的每个元素（从左到右）应用一个函数，将其减少为单个值。
- 返回值：累加后的值。
- 是否改变原数组：否。
- 示例：

```
1 var arr = [1, 2, 3, 4];
2 var sum = arr.reduce((acc, curr) => acc + curr, 0); // sum为10
3 console.log(sum); // 输出10
```

4.17. some()

- 语法：array.some(callback(element, index, array))
- 功能：测试数组中的至少一个元素是否满足由提供的函数定义的条件。
- 返回值：布尔值，如果至少有一个元素满足条件，则为true；否则为false。
- 是否改变原数组：否。
- 示例：

```
1 var arr = [1, 2, 3, 4];
2 var hasEven = arr.some(item => item % 2 === 0); // hasEven为true
3 console.log(hasEven); // 输出true
```

4.18. every()

- 语法：array.every(callback(element, index, array))

- **功能**：检查数组中的所有元素是否都满足由提供的函数定义的条件。
- **返回值**：布尔值，如果所有元素都满足条件，则为**true**；否则为**false**。
- **是否改变原数组**：否。
- **示例**：

JavaScript

```
1 var arr = [2, 4, 6, 8];
2 var allEven = arr.every(item => item % 2 === 0); // allEven为true
3 console.log(allEven); // 输出true
```

4.19. find()

- **语法**：array.find(callback(element, index, array))
- **功能**：返回数组中满足提供的测试函数的第一个元素的值。
- **返回值**：符合条件的第一个元素的值；如果没有符合条件的元素，则返回**undefined**。
- **是否改变原数组**：否。
- **示例**：

JavaScript

```
1 var arr = [1, 2, 3, 4];
2 var firstEven = arr.find(item => item % 2 === 0); // firstEven为2
3 console.log(firstEven); // 输出2
```

4.20. findIndex()

- **语法**：array.findIndex(callback(element, index, array))
- **功能**：返回数组中满足提供的测试函数的第一个元素的索引。
- **返回值**：符合条件的第一个元素的索引；如果没有符合条件的元素，则返回-1。
- **是否改变原数组**：否。
- **示例**：

```

1  var arr = [1, 3, 5, 7];
2  var index = arr.findIndex(item => item % 2 === 0); // index为-1
3  console.log(index); // 输出-1

```

4.21. fill()

- 语法：array.fill(value,start,end)
- 功能：将数组中所有元素或部分元素使用value值进行填充。如果出现负数表示从倒数开始
- 返回值：返回填充修改后的数组
- 是否改变原数组：是。
- 示例：

JSON

```

1  var arr = [1,2,3,1,2,3]
2  arr.fill("a")
3  console.log( arr ) // 填充全部 ['a', 'a', 'a', 'a', 'a', 'a']
4  arr.fill("ab",3)
5  console.log( arr ) // 从指定开始位置填充到最后 [1, 2, 3, 'ab', 'ab', 'ab']
6  arr.fill("aa",1,4)
7  console.log( arr ) // 从指定位置到结束位置进行填充 [1, 'aa', 'aa', 'aa', 2, 3]

```

4.22. copyWithin()

- 语法：array.copyWithin(index, start, end)
- 功能：将数组指定位置【start-end】的内容，复制到其他位置【index】（会覆盖原有内容）
 - index 从该位置开始替换内容。如果是负数表示倒数（必须的）
 - start 从该位置开始读取数据，默认是0。负数表示从末尾开始计算
 - end 从该位置停止读取数据，默认是数组长度。负数表示从末尾开始计算
 - 注意：如果这三个参数不是数值，会自动转成数值。
- 返回值：返回修改后的数组
- 是否改变原数组：是。
- 示例：

```

1  var arr = [1,2,3,4,5]
2  // 从下标3开始到末尾的值进行复制4,5, 复制到下标0的位置,覆盖了原来的1,2
3  arr.copyWithin(0,3)
4  console.log( arr ) // [4,5,3,4,5]
5
6  var arr = [1,5,9,3,5,7,4,5,6]
7  // 从下标3开始到下标4的值进行复制3,复制到下标1的位置,覆盖了原来的5
8  arr.copyWithin( 1 , 3 , 6 )
9  console.log( arr ) // [1, 3, 9, 3, 5, 7, 4, 5, 6]
10
11 var arr = [1,5,9,3,5,7,4,5,6]
12 // 从下标-3开始复制, 即4号位, 将4,5,6复制到下标3的位置, 覆盖了 3,5,7
13 arr.copyWithin(3,-3)
14 console.log( arr ) // [1,5,9,4,5,6,4,5,6]
15

```

4.23. at()

- 语法: `array.at(index)`
- 功能: 返回指定位置的数组项, 位置下标从0开始。
- 注意: 如果 `index`传入的参数是负数, 表示从倒数开始
- 返回值: 返回指定位置的数组项, 如果下标出界返回`undefined`。
- 是否改变原数组: 否。
- 示例:

```

1  var fruits = ["apple" , "banana" , "pear" ]
2  var item1 = fruits.at(1)
3  console.log( item1 ) // "banana"
4  var item2 = fruits.at(5)
5  console.log(item2)  // undefined
6  var item3 = fruits.at(-1)
7  console.log( item3 )  // "pear"

```

4.24. with()

- 语法: `array.with(index,value)`

- **功能：**通过指定下标修改元素的内容
- **返回值：**修改后的新数组
- **是否改变原数组：**否。
- **示例：**

```
1 var arr = [1,5,9,3,7];
2 // 将下标2的内容9修改为"hello"
3 var res = arr.with(2,"hello")
4 console.log( res ) // [1, 5, 'hello', 3, 7]
5 console.log( arr ) // [1, 5, 9, 3, 7]
```

4.25. flat() 重构flat

- **语法：**array.flat(deep)
- **功能：**扁平化数组。即 按照指定的深度递归遍历数组,并将所有元素与遍历的子数组中的元素合并到一个新数组中返回。默认扁平化一层。
- **返回值：**返回扁平化后的新数组。
- **是否改变原数组：**否。
- **示例：**

```
1 var arr [1,5,[9,[3,[5,7],4]],6,["a","b"]] // 多维数组
2 var resArr1 = arr.flat() // 默认扁平化一层
3 console.log( resArr1 ) // [1,5,9,[3,[5,7],4],6,"a","b"]
4 var resArr2 = arr.flat(2) // 深度为2 展开两层
5 console.log( resArr2 ) // [1,5,9,3,[5,7],4,6,"a","b"]
```

4.26. flatMap()

- **语法：**array.flatMap(callback(element, index, array))
- **功能：**先map再flat,只能扁平化一层
- **返回值：**返回操作扁平化后的新数组。
- **是否改变原数组：**否。
- **示例：**

```

1  var arr = [[1,2,3],[4,5,6],[7,8,[9]]];
2  var resArr = arr.flatMap(function(item){return item})
3  console.log( arr )    // [[1,2,3],[4,5,6],[7,8,[9]]]
4  console.log( resArr ) // [1,2,3,4,5,6,7,8,[9]]
5
6
7  var arr = [[1,2,3],[4,5],[["a"],7,8,9]]
8  var resArr = arr.flatMap(function(item){ return item.concat("hello") })
9  console.log( arr ) // [[1,2,3],[4,5],[["a"],7,8,9]]
10 console.log( resArr ) // [1,2,3,'hello',4,5,'hello',['a'],7,8,9,'hello']
11
12
13 var arr = [[1,2,3],[4,5],[["a"],7,8,9]]
14 var resArr = arr.flatMap(function(item){
15     return item.map(function(num , i){
16         return num * (i+1)
17     })
18 })
19 console.log( arr ) // [[1,2,3],[4,5],[["a"],7,8,9]]
20 console.log( resArr ) // [1, 4, 9, 4, 10, NaN, 14, 24, 36]
21

```

4.27. includes()

- 语法：array.includes(value)
- 功能：检测数组是否包含的值。
- 返回值：包含返回true，不否则回false。
- 是否改变原数组：否。
- 示例：

```

1  var arr = ["a","b",5,"c"]
2  console.log( arr.includes(5) ) // true
3  console.log( arr.includes("5") )// false

```

5. 静态方法

5.1. Array.isArray()

- 作用：检测某个值是不是数组
- 返回值：是数组返回true，否则返回false
- 示例：

```
1  var arr1 = [1,2,3];
2  console.log( Array.isArray( arr1 ) ) // true
3
4  var arr2 = "123";
5  console.log( Array.isArray(arr2) ) // false
```

5.2. Array.from()

- 语法：Array.from(arrayLike[,mapFn[,thisArg]])
 - arrayLike 要转换的对象(必须的)
 - mapFn 对每个元素进行操作，将操作后的结果返回在数组中（可选的）
 - thisArg 执行mapFn函数时的上下文对象（可选的）
- 作用：将类数组对象 或 可迭代对象 转成真正的数组
 - 类数组对象->具有length属性和数组键的对象
 - 可迭代对象->如：字符串、Set、Map等
- 返回值：新数组
- 示例：

```
1 // 类数组转真数组
2 function fn(){
3     console.log( arguments ) // Arguments ['a', 1, true, 156, 'hello']
4     console.log( Array.from( arguments ) ) // ['a', 1, true, 156, 'hello']
5 }
6 fn( "a",1,true,156,"hello" )
7
8 // 将可迭代字符串对象转换成数组
9 console.log( Array.from("hello") ) // ['h','e','l','l','o']
10
11 var arrayLike = { '0' : 5 , '1' : 9 , '2' : 2 , length : 3 };
12 var arr = Array.from( arrayLike )
13 console.log( arr ) // // [5,9,2]
14 var resArr = Array.from(arr , function(item , index){
15     return item * (index + 2)
16 })
17 console.log( resArr ) // [10,27,8] arr本身就是数组 相当于直接map操作
18 console.log( arr ) // [5,9,2]
```

6. 总结

1. 创建数组的两种方式，字面量和new关键字。
2. 通过下标访问和修改数组内容。
3. length 属性可以获取数组的长度。
4. 数组的常用方法 。